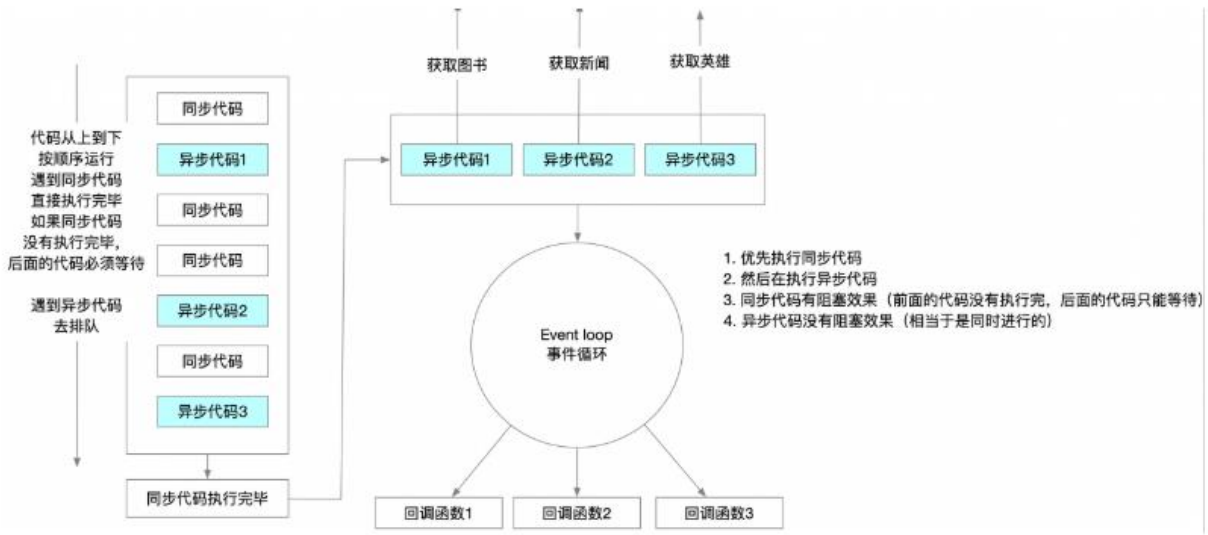
- **同步请求和异步请求**
  - JS代码分为同步代码和异步代码。目前，我们学习过的异步代码有：
    - 事件
    - 定时器
    - Ajax请求
  - 除此之外，其他所有代码都是同步代码。
  - 为什么要把代码分为同步和异步两类呢？因为它们的执行顺序有很大差别。
  - 假设有几段代码，既有同步代码，又有异步代码，他们的执行顺序如下：
    - 优先执行同步代码
    - 前一行同步代码没有执行完，后面的代码只能等待，这就是“阻塞”效果。
    - 遇到异步代码，去排队等待
    - 所有的同步代码执行完，才去检查是否有异步代码
    - 如果有异步代码，按顺序执行，但不会有“阻塞”效果。
  - 异步任务执行前，一般都会提前绑定一个回调函数
  - 当前的异步任务执行完毕，就会调用提前绑定的回调函数



- **JSON——JSON在Ajax中的作用**
  - JSON在Ajax请求的过程中，作用就是作为数据的载体。
  - 比如中国人和英国人交流，双方的语言不通，所以必须找一个翻译。
  - 服务端使用的编程语言可能是java、php等，前端使用的编程语言是JavaScript，双方的数据格式可能不一样，所以在交互数据的时候，得转换成双方都能识别的格式，比如JSON格式。

- **JSON——编写JSON**
  - JSON长得和JS数据差不多，但JSON是字符串类型。比如：

```
// JS 对象
var obj = { id: 1, name: 'zs' };

```

```
// JSON 字符串
var json = '{ "id": 1, "name": "zs" }';
```

- 编写JSON的具体要求：
  - 不能有 undefined
  - 不能有函数
  - 无论是属性名还是字符串类型的值，都必须加双引号。（单引号都不行）
  - 不允许写注释
- JSON中可以包括的数据类型
  - 数字
  - 字符串（必须加双引号）
  - 布尔
  - null
  - 数组
  - 对象

- **JSON——JSON和JS数据转换**
  - JSON ----> JS
    - var JS数据 = JSON.parse(JSON字符串);
    - 这个过程叫做 反序列化
  - JS ----> JSON
    - var JSON字符串 = JSON.stringify(JS数据);
    - 这个过程叫做 序列化

- **设置HTTP请求时限**
  - timeout -- Level 2 新增，IE8+支持，用于设置请求的超时时间，单位是毫秒
  - ontimeout -- Level 2 新增，IE10+支持，如果请求超时了，会触发 ontimeout 事件

```
var xhr = new XMLHttpRequest()
xhr.onload = function () {
    console.log(JSON.parse(this.response));
}
xhr.timeout = 30; // 单位是毫秒
// 当请求超时之后，会触发下面的函数
xhr.ontimeout = function () {
    alert('请求超时，请刷新重试');
}
xhr.open('GET', 'http://www.liulongbin.top:3006/api/getbooks');
```

```
xhr.send();
```

- **xhr2新增事件**
  - ontimeout -- 请求超时之后，触发的事件
  - onload -- 请求成功后触发的事件（因为已经表示成功后触发了，所以事件内部就不要加if判断了）
  - onerror -- 请求失败后触发的事件
  - onloadstart -- 请求开始的时候，触发的事件
  - onloadend -- 请求完成后触发的事件
  - onprogress -- 客户端下载（接收）数据的时候，触发的事件

- **xhr2新增的两个属性**
  - response -- 专门用于接收响应结果（可以接收任意类型的结果）
  - responseType -- 设置响应结果的类型
    - 默认是空，表示响应结果是字符串
    - json ，表示响应结果是json格式；如果设置为json，xhr对象内部就会自动的把JSON转成对象.
    - document，表示服务器返回的是xml

- **FormData对象——介绍**
  - Form是表单，Data是数据。猜测，它和表单数据有关系。
  - [FormData](https://developer.mozilla.org/zh-CN/docs/Web/API/FormData)是h5出现之后，新增的一个对象。用于管理表单数据。IE10+支持。
  - 可以这样理解，FormData的作用和 jQuery中的 serialize() 作用一样，用于快速收集表单数据，并且可以将结果直接提交给接口。
  - 创建的FormData对象，可直接通过 `xhr.send(FormData对象)` 提交给服务器的接口。

- **FormData对象——基本语法**

```
let fd = new FormData(myForm); // 参数是表单的DOM对象，可选
```

  - FormData的API：（除了append方法IE10支持外，其他方法IE均不支持）
    - `append('key', 'value');` -- 向对象中追加数据
    - `set('key', 'value');` -- 修改对象中的数据
    - delete('key'); -- 从对象中删除数据
    - get('key') -- 获取对象中的数据
    - getAll('key') -- 获取指定key的全部数据
    - forEach() -- 遍历对象中的数据

```html
<form action="">
    <input type="text" name="username"><br />
    <input type="password" name="pwd"><br />
    <input type="radio" name="sex" value="nan">
    <input type="radio" name="sex" value="nv"><br />

    <button>提交</button>
</form>

<script>
    document.querySelector('form').onsubmit = function (e) {
        e.preventDefault();

        // 使用FormData收集表单数据
        var fd = new FormData(this); // 传入表单的DOM对象

        // append向fd对象中追加数据
        fd.append('age', 20); // 追加一个age
        fd.append('username', 'lisi'); // 追加一个username

        // set修改fd中的数据
        fd.set('sex', 'yao'); // 修改sex

        // get用于获取一个值
        console.log(  fd.get('username')  ); // 获取到一个username的值
        console.log(  fd.getAll('username')  ); // 获取到全部username的值

        // console.log(fd); // 输出fd没用，看不到数据
        // 只能通过forEach来检查对象中有哪些数据
        fd.forEach(function (val, key) {
            // console.log(key, val);
        });

    }
</script>
```

- **FormData对象——FormData配合XHR对象**
  - FormData本就是配合XHR对象一起使用的
  - 前面我们收集到了很多值，现在，我们可以通过Ajax，将这些值提交给接口。

- 只能通过POST方式提交FormData对象
- 不能设置Content-Type请求头，因为当提交FormData的时候，XHR对象会自动设置这个请求头。

```
<form action="">
    姓名: <input type="text" name="username"><br>
    年龄: <input type="text" name="age"><br>
    身高: <input type="text" name="height"><br>
    <button>提交</button>
</form>

<script>

    document.querySelector('form').onsubmit = function (e) {
        e.preventDefault();
        var fd = new FormData(this);

        // ajax提交数据到接口
        var xhr = new XMLHttpRequest();
        xhr.onload = function () {
            console.log(this.response);
        }
        xhr.open('POST', 'http://www.liulongbin.top:3006/api/formdata');
        // xhr.setRequestHeader(); // 使用FormData，不要设置请求头；写了请求头，反而会报错
        xhr.send(fd);
    }
</script>
```

- **FormData对象——FormData配合jQuery使用**

```
<form action="">
    姓名: <input type="text" name="username"><br>
    年龄: <input type="text" name="age"><br>
    身高: <input type="text" name="height"><br>
    <button>提交</button>
</form>

<script src="./jquery.js"></script>
```

```
<script>
    $('form').on('submit', function (e) {
        e.preventDefault();
        // 使用FormData收集数据
        var fd = new FormData(this); // 传入DOM对象哟~~~
        // 使用$.ajax()提交
        $.ajax({
            url: 'http://www.liulongbin.top:3006/api/formdata',
            type: 'POST',
            data: fd, // 这里直接使用FormData
            processData: false, // 必填项
            contentType: false, // 必填项
            success: function (res) {
                console.log(res);
            }
        })
    })
</script>
```

- processData：
  - 前文讲，jQuery默认会把data转换成查询字符串格式，这里 `processData: false`，表示不要把FormData对象转换成查询字符串。因为原生（底层）实现是 `xhr.send(fd)`，也是直接提交FormData对象的。
- contentType:
  - 前文讲，提交FormData，不能自己设置Content-Type这个请求头，这里 `contentType: false`，表示不要设置这个请求头。

- **FormData对象——使用FormData的注意事项**
  - 使用FormData，要求表单各项必须有name属性，因为FormData也是根据表单各项的name属性获取值的
  - 实例化 FormData对象，传入表单的DOM对象，可以快速收集到表单各项值。
  - 可以收集文件信息，这是和 serialize 不一样的。可以完成文件上传。
  - 如果要检查FormData中有哪些值，需要使用forEach遍历。
  - 如果需要动态添加或修改FormData中的值，可以调用 FormData的append或set方法。

- **FormData对象——FormData和serialize的区别**
  - FormData属于原生的代码；serialiaze和serializeArray是jQuery封装的方法
  - 都需要设置表单各项的name属性。
  - jQuery中提交FormData，必须指定 processData: false 和 contentType: false。
  - FormData可以收集文件域的值，而serialize不能。也就是说，如果有文件上传，必须使用FormData。
  - 得到的结果的数据类型不一样